

Chapitre III : Diagramme de classes

1. Introduction

Le diagramme de classes est considéré comme le plus important de la modélisation orientée objet, il est le seul obligatoire lors d'une telle modélisation.

Alors que le diagramme de cas d'utilisation montre un système du point de vue des acteurs, le diagramme de classes en montre la structure interne. Il permet de fournir une représentation abstraite des objets du système qui vont interagir pour réaliser les cas d'utilisation.

Il s'agit d'une vue statique, car on ne tient pas compte du facteur temporel dans le comportement du système. permet de modéliser les classes du système et leurs relations indépendamment d'un langage de programmation particulier.

Les principaux éléments de cette vue statique sont les classes et leurs relations : association, généralisation...etc.

2. Les classes

2.1. Notions de classe et d'instance de classe

Une *instance* est une concrétisation d'un concept abstrait. Par exemple :

- la voiture Rav4 qui se trouve dans votre garage est une instance du concept abstrait *Automobile* ;
- l'amitié qui lie Jean et Marie est une instance du concept abstrait *Amitié* ;

Une classe est un concept abstrait représentant des éléments variés comme :

- des éléments concrets (ex. : des avions),
- des éléments abstraits (ex. : des commandes de marchandises ou services),
- des éléments comportementaux (ex. : des tâches), etc.

Tout système orienté objet est organisé autour des classes. Une classe est la description formelle d'un ensemble d'objets ayant une sémantique et des caractéristiques communes.

Un objet est une instance d'une classe. C'est une entité discrète dotée d'une identité, d'un état et d'un comportement que l'on peut invoquer. Les objets sont des éléments individuels d'un système en cours d'exécution.

Par exemple, si l'on considère que *Homme* (au sens être humain) est un concept abstrait, on peut dire que la personne Ahmed est une instance de Homme. Si *Homme* était une classe, Ahmed en serait une instance : un objet.

2.2. Caractéristiques d'une classe

Une classe définit un jeu d'objets dotés de caractéristiques communes. Les caractéristiques d'un objet permettent de spécifier son *état* et son *comportement*. Les caractéristiques d'un objet étaient soit des attributs, soit des opérations. Ce n'est pas exact dans un diagramme de classe, car les terminaisons d'associations sont des propriétés qui peuvent faire partie des caractéristiques d'un objet au même titre que les attributs et les opérations.

État d'un objet :

- ce sont les attributs et généralement les *terminaisons d'associations*, tous deux réunis sous le terme de *propriétés structurelles*, ou tout simplement *propriétés*, qui décrivent l'état d'un objet. Les attributs sont utilisés pour des valeurs de données pures, dépourvues d'identité, telles que les nombres et les chaînes de caractères. Les associations sont utilisées pour connecter les classes du diagramme de classe ; dans ce cas, la terminaison de l'association (du côté de la classe cible) est généralement une propriété de la classe de base. Les propriétés décrites par les attributs prennent des valeurs lorsque la classe est instanciée. L'instance d'une association est appelée un lien.

Comportement d'un objet :

- les opérations décrivent les éléments individuels d'un comportement que l'on peut invoquer. Ce sont des fonctions qui peuvent prendre des valeurs en entrée et modifier les attributs ou produire des résultats. Une opération est la spécification d'une méthode.

Les attributs, les terminaisons d'association et les méthodes constituent donc les caractéristiques d'une classe (et de ses instances).

2.3. Représentation graphique

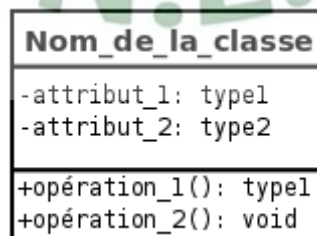


Figure 1 : Représentation UML d'une classe.

Une classe est un classeur. Elle est représentée par un rectangle divisé en trois compartiments. Le premier indique le nom de la classe, le deuxième ses attributs et le troisième ses opérations.

2.4. Encapsulation, visibilité

ClasseX
-attribut_1: int -attribut_2: string
+set_attribut_1(int): void +get_attribut_1(): int +set_attribut_2(string): void +get_attribut_2(): string

Figure 2 : Bonnes pratiques concernant la manipulation des attributs.

L'encapsulation est un mécanisme consistant à rassembler les données et les méthodes au sein d'une structure en cachant l'implémentation de l'objet, c'est-à-dire en empêchant l'accès aux données par un autre moyen que les services proposés. Ces services accessibles (offerts) aux utilisateurs de l'objet définissent ce que l'on appelle l'interface de l'objet (sa vue externe). L'encapsulation permet donc de garantir l'intégrité des données contenues dans l'objet.

L'encapsulation permet de définir des niveaux de visibilité des éléments d'un conteneur. La visibilité déclare la possibilité pour un élément de modélisation de référencer un élément qui se trouve dans un espace de noms différent de celui de l'élément qui établit la référence. Elle fait partie de la relation entre un élément et le conteneur qui l'héberge, ce dernier pouvant être un paquetage, une classe ou un autre espace de noms. Il existe quatre visibilités prédéfinies.

Public ou + : tout élément qui peut voir le conteneur peut également voir l'élément indiqué.

Protected ou # : seul un élément situé dans le conteneur ou un de ses descendants peut voir l'élément indiqué.

Private ou - : seul un élément situé dans le conteneur peut voir l'élément.

Package ou ~ ou rien : seul un élément déclaré dans le même paquetage peut voir l'élément.

Dans la pratique, lorsque des attributs doivent être accessibles de l'extérieur, il est préférable que cet accès ne soit pas direct, mais se fasse par l'intermédiaire d'opérations.

2.5. Nom d'une classe

Le nom de la classe doit évoquer le concept décrit par la classe. Il commence par une majuscule. Pour indiquer qu'une classe est abstraite, il faut ajouter le mot-clé *abstract*.

2.6. Les attributs

2.6.1. Attributs de la classe

Les attributs définissent des informations qu'une classe ou un objet doivent connaître. Ils représentent les données encapsulées dans les objets de cette classe. Chacune de ces informations est définie par un nom, un type de données, une visibilité et peut être initialisée. Le nom de l'attribut doit être unique dans la classe. La syntaxe de la déclaration d'un attribut est la suivante :

```
<visibilité> [/] <nom_attribut> : <type> [ '['<multiplicité>' ] [{<contrainte>}] ] [ =  
<valeur_par_défaut> ]
```

Le type de l'attribut (<type>) peut être un nom de classe ou un type de donnée prédéfini. La multiplicité (<multiplicité>) d'un attribut précise le nombre de valeurs que l'attribut peut contenir.

2.6.2. Attributs de classe

Par défaut, chaque instance d'une classe possède sa propre copie des attributs de la classe. Les valeurs des attributs peuvent donc différer d'un objet à un autre. Cependant, il est parfois nécessaire de définir un *attribut de classe* (*static* en Java ou en C++) qui garde une valeur unique et partagée par toutes les instances de la classe. Les instances ont accès à cet attribut, mais n'en possèdent pas une copie. Un attribut de classe n'est donc pas une propriété d'une instance, mais une propriété de la classe et l'accès à cet attribut ne nécessite pas l'existence d'une instance.

Graphiquement, un attribut de classe est souligné.

2.6.3. Attributs dérivés

Les attributs dérivés peuvent être calculés à partir d'autres attributs et de formules de calcul. Lors de la conception, un attribut dérivé peut être utilisé comme marqueur jusqu'à ce que vous puissiez déterminer les règles à lui appliquer.

Les attributs dérivés sont symbolisés par l'ajout d'un « / » devant leur nom.

2.7. Les méthodes

2.7.1. Méthode de la classe

Dans une classe, une opération (même nom et mêmes types de paramètres) doit être unique. Quand le nom d'une opération apparaît plusieurs fois avec des paramètres différents, on dit que l'opération est surchargée. En revanche, il est impossible que deux opérations ne se distinguent que par leur valeur retournée.

La déclaration d'une opération contient les types des paramètres et le type de la valeur de retour, sa syntaxe est la suivante :

```
<visibilité> <nom_méthode> ([<paramètre_1>, ... , <paramètre_N>]) :  
[<type_renvoyé>] [{<propriétés>}]
```

Les propriétés (<propriétés>) correspondent à des contraintes ou à des informations complémentaires comme les exceptions, les préconditions, les postconditions ou encore l'indication qu'une méthode est abstraite (mot-clef *abstract*), etc.

2.7.2. Méthode de classe

Comme pour les attributs de classe, il est possible de déclarer des méthodes de classe. Une méthode de classe ne peut manipuler que des attributs *de classe* et ses propres paramètres. Cette méthode n'a pas accès aux attributs *de la classe* (i.e. des instances de la classe). L'accès à une méthode de classe ne nécessite pas l'existence d'une instance de cette classe.

Graphiquement, une méthode de classe est soulignée.

3. Relations entre classes

3.1. Notion d'association

Une association est une relation entre deux classes (association binaire) ou plus (association n-aire), qui décrit les connexions structurelles entre leurs instances. Une association indique donc qu'il peut y avoir des liens entre des instances des classes associées.

Comment une association doit-elle être modélisée ? Plus précisément, quelle différence existe-t-il entre les deux diagrammes de la figure 3 ?

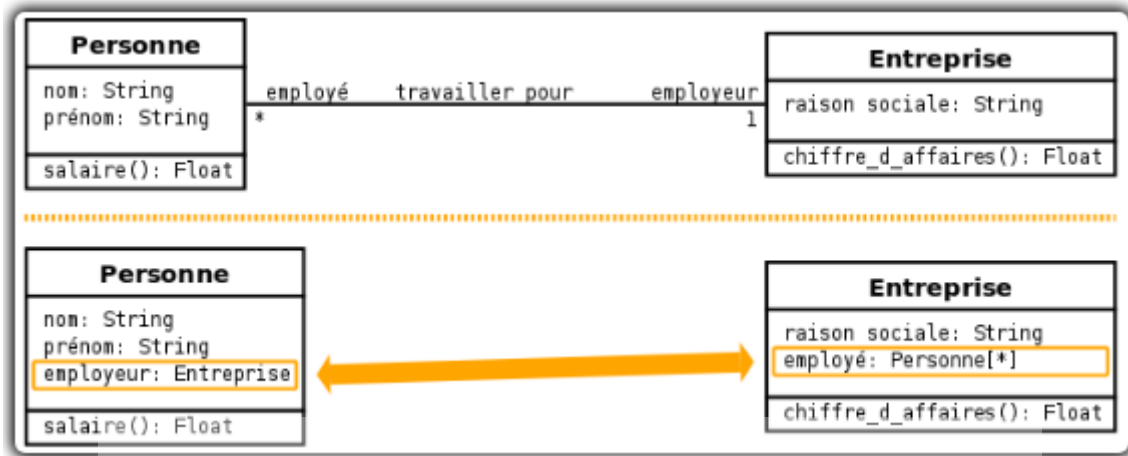


Figure 3 : Deux façons de modéliser une association.

Dans la première version, l'association apparaît clairement et constitue une entité distincte. Dans la seconde, l'association se manifeste par la présence de deux attributs dans chacune des classes en relation. C'est en fait la manière dont une association est généralement implémentée dans un langage objet quelconque, mais pas dans tout langage de représentation).

Les terminaisons d'associations et les attributs sont deux éléments conceptuellement très proches que l'on regroupe sous le terme de *propriété*.

3.2. Terminaison d'association

La possession d'une terminaison d'association par le classeur situé à l'autre extrémité de l'association peut être spécifiée graphiquement par l'adjonction d'un petit cercle plein (point) entre la terminaison d'association et la classe (cf. figure 4). Il n'est pas indispensable de préciser la possession des terminaisons d'associations. Dans ce cas, la possession est ambiguë. Par contre, si l'on indique des possessions de terminaisons d'associations, toutes les terminaisons d'associations sont non ambiguës : la présence d'un point implique que la terminaison d'association appartient à la classe située à l'autre extrémité, l'absence du point implique que la terminaison d'association appartient à l'association.

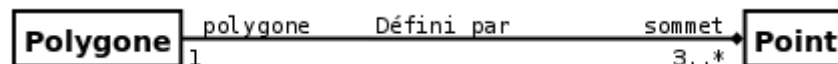


Figure 4 : Utilisation d'un petit cercle plein pour préciser le propriétaire d'une terminaison d'association

Par exemple, le diagramme de la figure 4 précise que la terminaison d'association *sommet* (i.e. la propriété *sommet*) appartient à la classe *Polygone* tandis que la terminaison d'association *polygone* (i.e. la propriété *polygone*) appartient à l'association *Défini par*.

3.3. Association binaire et n-aire

3.3.1. Association binaire

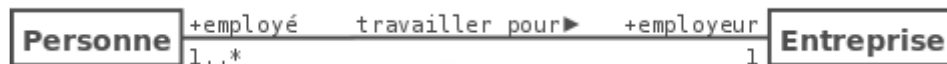


Figure 5 : Exemple d'association binaire.

Une association binaire est matérialisée par un trait plein entre les classes associées (cf. figure 5). Elle peut être ornée d'un nom, avec éventuellement une précision du sens de lecture (► ou ◄).

Quand les deux extrémités de l'association pointent vers la même classe, l'association est dite *réflexive*.

3.3.2. Association n-aire

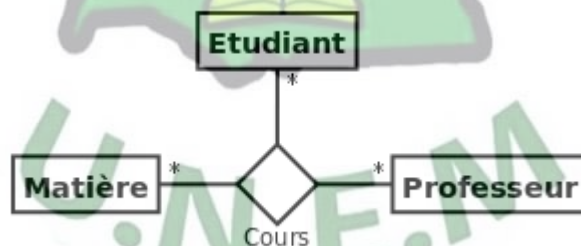


Figure 6 : Exemple d'association n-aire.

On représente une association n-aire par un grand losange avec un chemin partant vers chaque classe participante (cf. figure 6). Le nom de l'association, le cas échéant, apparaît à proximité du losange.

3.4. Multiplicité ou cardinalité

La multiplicité associée à une terminaison d'association, d'agrégation ou de composition déclare le nombre d'objets susceptibles d'occuper la position définie par la terminaison d'association. Voici quelques exemples de multiplicité :

- exactement un : 1 ou 1..1 ;
- plusieurs : * ou 0..* ;
- au moins un : 1..* ;
- de un à six : 1..6.

Dans une association binaire (cf. figure 5), la multiplicité sur la terminaison cible contraint le nombre d'objets de la classe cible pouvant être associés à un seul objet donné de la classe source (la classe de l'autre terminaison de l'association).

Dans une association n-aire, la multiplicité apparaissant sur le lien de chaque classe s'applique sur une instance de chacune des classes, à l'exclusion de la classe-association et de la classe considérée. Par exemple, si on prend une association ternaire entre les classes (A, B, C), la multiplicité de la terminaison C indique le nombre d'objets C qui peuvent apparaître dans l'association avec une paire particulière d'objets A et B.

3.5. Classe-association

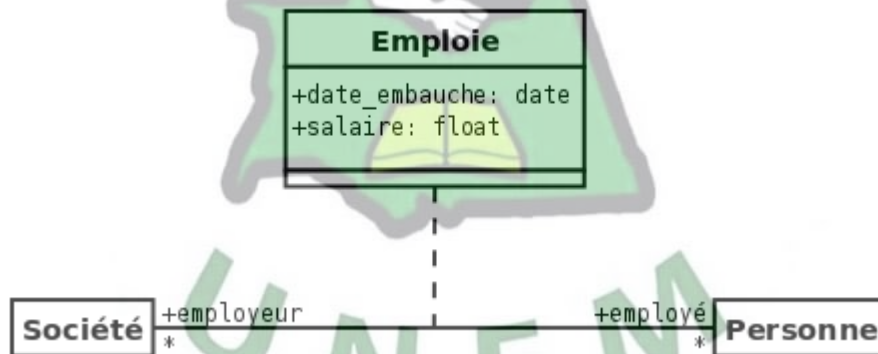


Figure 7 : Exemple de classe-association

Parfois, une association doit posséder des propriétés. Par exemple, l'association *Emploie* entre une société et une personne possède comme propriétés le salaire et la date d'embauche. En effet, ces deux propriétés n'appartiennent ni à la société, qui peut employer plusieurs personnes, ni aux personnes, qui peuvent avoir plusieurs emplois. Il s'agit donc bien de propriétés de l'association *Emploie*. Les associations ne pouvant posséder de propriété, il faut introduire un nouveau concept pour modéliser cette situation : celui de *classe-association*.

Une classe-association possède les caractéristiques des associations et des classes : elle se connecte à deux ou plusieurs classes et possède également des attributs et des opérations.

Une classe-association est caractérisée par un trait discontinu entre la classe et l'association qu'elle représente (figure 10).

3.6. Agrégation et composition

3.6.1. Agrégation



Figure 8 : Exemple de relation d'agrégation et de composition.

Une association simple entre deux classes représente une relation structurelle entre pairs, c'est-à-dire entre deux classes de même niveau conceptuel : aucune des deux n'est plus importante que l'autre.

Lorsque l'on souhaite modéliser une relation *tout/partie* où une classe constitue un élément plus grand (*tout*) composé d'éléments plus petits (*partie*), il faut utiliser une agrégation.

Une agrégation est une association qui représente une relation d'inclusion structurelle ou comportementale d'un élément dans un ensemble. Graphiquement, on ajoute un losange vide (♦) du côté de l'agrégat (cf. figure 8).

3.6.2. Composition

La composition, également appelée agrégation composite, décrit une contenance structurelle entre instances. Ainsi, la destruction de l'objet composite implique la destruction de ses composants.

Une instance de la partie appartient toujours à au plus une instance de l'élément composite : la multiplicité du côté composite ne doit pas être supérieure à 1 (*i.e.* 1 ou 0..1). Graphiquement, on ajoute un losange plein (◆) du côté de l'agrégat (cf. figure 8).

3.7. Généralisation et Héritage

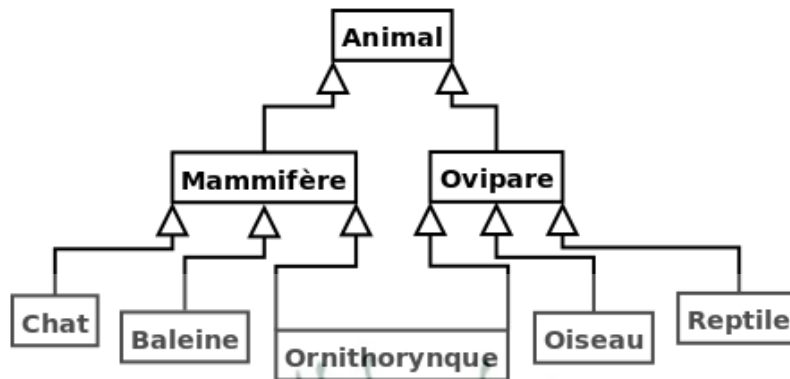


Figure 9 : Partie du règne animal décrit avec l'héritage multiple.

La généralisation décrit une relation entre une classe générale (classe de base ou classe parent) et une classe spécialisée (sous-classe). La classe spécialisée est intégralement cohérente avec la classe de base, mais comporte des informations supplémentaires (attributs, opérations, associations). Un objet de la classe spécialisée peut être utilisé partout où un objet de la classe de base est autorisé.

Dans le langage UML, ainsi que dans la plupart des langages objet, cette relation de généralisation se traduit par le concept d'héritage. On parle également de relation d'héritage. Ainsi, l'héritage permet la classification des objets (cf. figure 9).

Le symbole utilisé pour la relation d'héritage ou de généralisation est une flèche avec un trait plein dont la pointe est un triangle fermé désignant le cas le plus général (cf. figure 9).

Les propriétés principales de l'héritage sont :

- la classe enfant possède toutes les caractéristiques de ses classes parents, mais elle ne peut accéder aux caractéristiques privées de cette dernière ;
- une classe enfant peut redéfinir (même signature) une ou plusieurs méthodes de la classe parent. Sauf indication contraire, un objet utilise les opérations les plus spécialisées dans la hiérarchie des classes ;
- toutes les associations de la classe parent s'appliquent aux classes dérivées ;
- une instance d'une classe peut être utilisée partout où une instance de sa classe parent est attendue. Par exemple, en se basant sur le diagramme de la figure 9, toute opération acceptant un objet d'une classe *Animal* doit accepter un objet de la classe *Chat* ;

- une classe peut avoir plusieurs parents, on parle alors d'héritage multiple (cf. la classe *Ornithorynque* de la figure 9). Le langage C++ est un des langages objet permettant son implémentation effective, le langage Java ne le permet pas.

En UML, la relation d'héritage n'est pas propre aux classes. Elle s'applique à d'autres éléments du langage comme les acteurs ou les cas d'utilisation.

3.8. Dépendance



Figure 10 : Exemple de relation de dépendance

Une dépendance est une relation unidirectionnelle exprimant une dépendance sémantique entre des éléments du modèle. Elle est représentée par un trait discontinu orienté (cf. figure 10). Elle indique que la modification de la cible peut impliquer une modification de la source. La dépendance est souvent stéréotypée pour mieux expliciter le lien sémantique entre les éléments du modèle (cf. figure 10).

On utilise souvent une dépendance quand une classe en utilise une autre comme argument dans la signature d'une opération. Par exemple, le diagramme de la figure 10 montre que la classe *Confrontation* utilise la classe *Stratégie*, car la classe *Confrontation* possède une méthode *confronter* dont deux paramètres sont du type *Stratégie*. Si la classe *Stratégie* change, alors des modifications devront également être apportées à la classe *Confrontation*.

4. Diagramme d'objets

4.1. Présentation

Un diagramme d'objets représente des objets (*i.e.* instances de classes) et leurs liens (*i.e.* instances de relations) pour donner une vue figée de l'état d'un système à un instant donné. Un diagramme d'objets peut être utilisé pour :

- illustrer le modèle de classes en montrant un exemple qui explique le modèle ;
- préciser certains aspects du système en mettant en évidence des détails imperceptibles dans le diagramme de classes ;
- prendre une image (*snapshot*) d'un système à un moment donné.

Le diagramme de classes modélise les règles et le diagramme d'objets modélise des faits.

Par exemple, le diagramme de classes de la figure 11 montre qu'une entreprise emploie au moins deux personnes et qu'une personne travaille dans au plus deux entreprises. Le diagramme d'objets modélise ici une entreprise particulière (*PERTNE*) qui emploie trois personnes.

4.2. Représentation

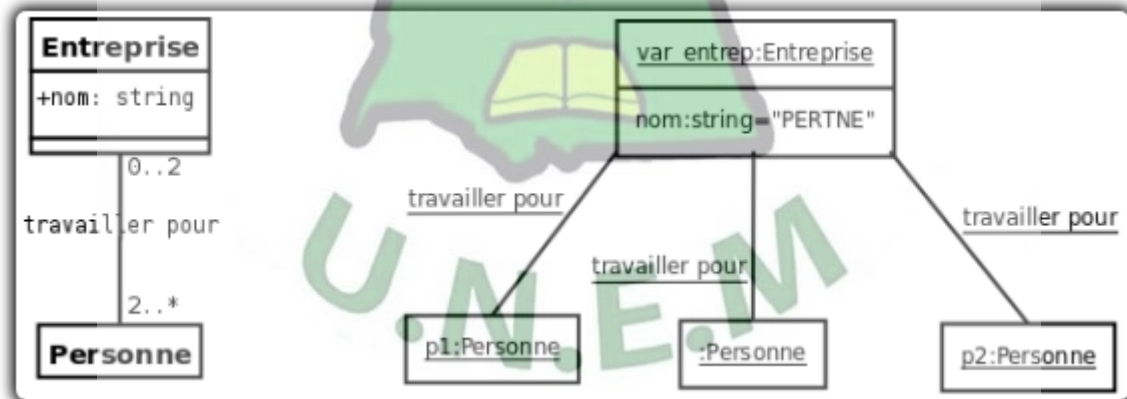


Figure 11 : Exemple de diagramme de classes et de diagramme d'objets associé

Graphiquement, un objet se représente comme une classe. Cependant, le compartiment des opérations n'est pas utile. De plus, le nom de la classe dont l'objet est une instance est précédé d'un << : >> et est souligné. Pour différencier les objets d'une même classe, leur identifiant peut être ajouté devant le nom de la classe. Enfin les attributs reçoivent des valeurs. Quand certaines valeurs d'attribut d'un objet ne sont pas renseignées, on dit que l'objet est partiellement défini.

5. Élaboration d'un diagramme de classes

Une démarche couramment utilisée pour bâtir un diagramme de classes consiste à :

Trouver les classes du domaine étudié.

- Cette étape empirique se fait généralement en collaboration avec un expert du domaine. Les classes correspondent généralement à des concepts ou des substantifs du domaine ;

Trouver les associations entre classes.

- Les associations correspondent souvent à des verbes, ou des constructions verbales, mettant en relation plusieurs classes, comme << est composé de >>, << pilote >>, << travaille pour >>.

Trouver les attributs des classes.

- Les attributs correspondent souvent à des substantifs, ou des groupes nominaux, tels que << la masse d'une voiture >> ou << le montant d'une transaction >>. Les adjectifs et les valeurs correspondent souvent à des valeurs d'attributs. Vous pouvez ajouter des attributs à toutes les étapes du cycle de vie d'un projet (implémentation comprise). N'espérez pas trouver tous les attributs dès la construction du diagramme de classes ;

Organiser et simplifier le modèle.

- En éliminant les classes redondantes et en utilisant l'héritage ;

Itérer et raffiner le modèle.

- Un modèle est rarement correct dès sa première construction. La modélisation objet est un processus non pas linéaire, mais itératif.